# DevDocs — Project / Workspace Management

## 1. Purpose

Project / Workspace Management allows users to organize their uploaded codebases and documentation into separate projects.

Instead of treating every uploaded file as an independent document, DevDocs should understand that the files belong to a particular **project**.

For example:

```
DevDocs
│
├── E-Commerce API
│   ├── backend/
│   ├── frontend/
│   ├── README.md
│   └── ...
│
├── Banking Application
│   ├── src/
│   ├── docs/
│   └── ...
│
└── Personal AI Project
    ├── app/
    ├── requirements.txt
    └── README.md
```

Each project becomes an isolated workspace for its own files, ingestion state, and conversations.

---

# 2. Why Projects / Workspaces Are Needed

Without project separation, DevDocs would have difficulty answering questions when multiple codebases are uploaded.

For example, suppose the user uploads:

```
Project A → E-Commerce application
Project B → Banking application
```

If both projects share the same document store, a query such as:

"How is authentication implemented?"

could potentially retrieve information from both applications.

Instead, DevDocs should know:

```
Current Project:
E-Commerce Application
```

and restrict retrieval to that project.

Therefore:

> **A project/workspace acts as the boundary for files, metadata, ingestion, and RAG retrieval.**

---

# 3. Core Project Features

The initial Project / Workspace Management feature should support:

### Create Project

The user can create a project with basic information such as:

```
Project Name
Description
```

Example:

```
Project Name:
DevDocs

Description:
AI-powered developer documentation assistant
```

---

### View Projects

The user should have a project list/dashboard.

Example:

```
MY PROJECTS

┌──────────────────────────────────────┐
│ E-Commerce API                       │
│ 42 files                             │
│ Last updated: Today                  │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ Banking Application                  │
│ 86 files                             │
│ Last updated: Yesterday              │
└──────────────────────────────────────┘
```

---

### Open Project

Selecting a project opens its workspace.

Example:

```
E-Commerce API

Files
Documents
Chat
Search
```

All operations inside this workspace should operate on that project.

---

### Rename Project

The user should be able to change the project name.

Example:

```
Old:
My Project

New:
E-Commerce Backend
```

---

### Delete Project

Deleting a project should remove its associated project data.

This includes:

- Project metadata
- Uploaded file metadata
- Ingestion information
- Chunks
- Embeddings
- Conversations associated with the project

The UI should ask for confirmation before deletion.

---

# 4. Project as the Main Boundary

The project ID becomes an important identifier throughout the backend.

For example:

```
Project
   ↓
project_id = 123
```

Files associated with the project:

```
file_id   project_id   file_path
1         123          README.md
2         123          backend/main.py
3         123          backend/auth.py
4         123          frontend/App.jsx
```

Chunks should also retain the project association:

```
chunk_id
project_id
file_id
content
embedding
metadata
```

This becomes particularly important for RAG.

When the user asks:

"How does authentication work?"

DevDocs should effectively retrieve:

```
WHERE project_id = current_project
```

before performing semantic retrieval.

This prevents information from unrelated projects from entering the answer.

---

# 5. Relationship With File Upload

Project management and file ingestion work together.

The intended flow is:

```
Create Project
      ↓
Open Project
      ↓
Upload Project Files
      ↓
File Discovery
      ↓
File Filtering
      ↓
Change Detection
      ↓
Parsing
      ↓
Chunking
      ↓
Embedding
      ↓
Vector Database
```

The important distinction is:

> **Project Management decides WHERE the files belong.**
>
> **Ingestion decides HOW those files are processed.**

---

# 6. Project Overview

The project workspace can display basic information about the uploaded project.

For V1, this should remain simple.

Example:

```
PROJECT OVERVIEW

Project:
E-Commerce API

Files:
42

Documentation:
8

Source Files:
29

Other Files:
5

Last Ingested:
12 Sep 2026

Status:
Ready
```

The overview should be based on information already available from file discovery and ingestion.

---

# 7. Language Detection

A useful addition to the Project Overview is basic **language/file-type information**.

This does not require an LLM.

It can be determined from file extensions.

Example:

```
LANGUAGES

Java        24 files
```

```
Python      12 files
JavaScript   8 files
TypeScript   5 files
Markdown     4 files
```

A GitHub-like visual representation could eventually be used.

For example:

```
Java        ████████████████
Python      ████████
JavaScript  ████
TypeScript  ███
Markdown    ██
```

The exact visualization can be decided during frontend implementation.

---

# 8. What We Are NOT Doing in V1

We originally considered detecting frameworks and technologies such as:

```
Spring Boot
React
FastAPI
Django
Express
```

by inspecting dependency files, configuration files, project structure, and source-code patterns.

However, this is **not part of the current V1 scope**.

We will keep framework/technology detection as a **future enhancement**.

Therefore, V1 will not attempt to determine:

```
Spring Boot
React
FastAPI
Django
Next.js
etc.
```

from the uploaded codebase.

This keeps the first implementation simpler and allows us to focus on the core DevDocs workflow.

---

# 9. Future Enhancement — Technology Detection

Framework and technology detection can be added later as a separate Project Intelligence feature.

Possible future detection:

```
Languages
├── Java
├── Python
├── JavaScript
└── TypeScript

Frameworks
├── Spring Boot
├── React
├── FastAPI
└── Django

Databases
├── PostgreSQL
├── MySQL
└── MongoDB

Infrastructure
├── Docker
└── Kubernetes
```

The planned approach would be deterministic/rule-based rather than LLM-based.

For example:

```
pom.xml
   +
spring-boot dependency
   +
@SpringBootApplication
   ↓
Spring Boot detected
```

Similarly:

```
package.json
   +
react dependency
```

```
   +
.jsx/.tsx files
   ↓
React detected
```

However, this is intentionally deferred.

---

# 10. Re-uploading a Project

Users may modify their project after the initial upload.

For example:

```
First upload:

backend/main.py
backend/auth.py
README.md
```

Later:

```
backend/main.py      ← modified
backend/auth.py      ← unchanged
backend/users.py     ← new
README.md             ← unchanged
```

The user can upload the updated project again.

DevDocs should not blindly process every file again.

Instead:

```
New Upload
    ↓
File Discovery
    ↓
Compare With Existing Project
    ↓
Detect Changes
    ↓
Process Only Necessary Files
```

This uses the file-level change detection system already defined for ingestion.

---

# 11. Project Overview After Re-upload

The Project Overview should automatically reflect the current project state.

For example:

```
Before:

Python      10 files
JavaScript   5 files
Total       15 files
```

After the user adds three Python files:

```
After:

Python      13 files
JavaScript   5 files
Total       18 files
```

This update is computationally inexpensive because basic project statistics can be calculated from file metadata.

We do not need to send the project to an LLM.

---

# 12. Computational Cost

Project management itself is extremely lightweight.

Operations such as:

```
Create project
Rename project
Delete project
Count files
Count file types
Calculate file statistics
```

are ordinary database/file operations.

They do not require:

- LLM calls
- embeddings

- vector search
- model inference

Therefore, these operations remain compatible with the project's:

**₹0 cost requirement.**

---

# 13. Suggested Backend Structure

Conceptually, the backend can contain:

```
FastAPI
│
├── Projects
│   ├── Create project
│   ├── Get projects
│   ├── Get project
│   ├── Update project
│   └── Delete project
│
├── Files
│   ├── Upload
│   ├── List files
│   └── File metadata
│
├── Ingestion
│   ├── Change detection
│   ├── Parsing
│   ├── Chunking
│   └── Embedding
│
└── Chat
    └── Project-scoped RAG
```

---

# 14. High-Level Data Relationship

A simplified relationship can be:

```
Project
   │
   ├── Files
   │      │
   │      ├── File metadata
   │      ├── File hash
   │      └── Ingestion status
```

```
    │
    ├── Chunks
    │       │
    │       └── Embeddings
    │
    └── Conversations
```

The `project_id` connects these entities.

---

# 15. V1 Scope

The current V1 Project / Workspace Management feature includes:

### Project Management

- Create project
- View projects
- Open project
- Rename project
- Delete project

### Project Overview

- Project name
- File count
- Basic file statistics
- Language/file-type breakdown
- Ingestion status
- Last updated/ingested information

### Project Isolation

- Files belong to a project
- Chunks belong to a project
- Conversations belong to a project
- RAG retrieval is scoped to the current project

### Re-upload Support

- Re-upload updated project
- Detect new files
- Detect modified files
- Detect deleted files
- Skip unchanged files
- Update project statistics

---

# 16. Explicitly Deferred

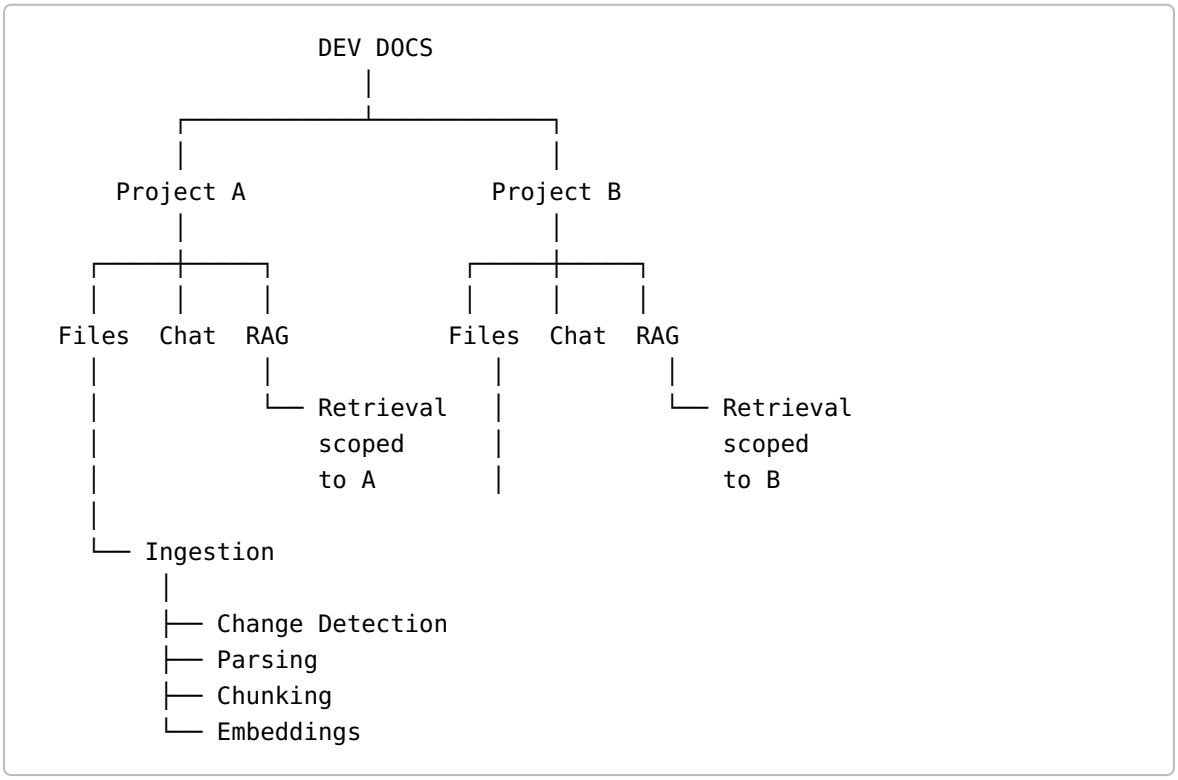
The following are **future enhancements**, not V1:

- Framework detection
- Technology detection
- Database detection
- Infrastructure detection
- Dependency graph
- Advanced project intelligence
- GitHub integration
- Automatic Git synchronization
- Branch-aware projects
- Project version history

---

# 17. Final Mental Model

The Project / Workspace feature can be thought of as the **container around everything DevDocs knows about one software project**.




The core principle is:

> **A workspace keeps one project's files, knowledge, conversations, and RAG context isolated from every other project.**

This gives DevDocs a clean foundation for the features that come later.